S Vara Prasad Reddy – Roll No: B23CM1057 – NLU

1. Corpus Curation & Preprocessing

To train our Word2Vec models, we curated a custom corpus from IIT Jodhpur sources (department pages, academic regulations, research highlights). The preprocessing pipeline followed these exact steps:

Step-1: Data Collection: Scraped raw HTML content from the official web pages.

Step-2: Boilerplate Removal: Removed structural HTML tags, scripts, footers, and standard web navigation text (e.g., "Skip to main content") using BeautifulSoup.

Step-3: Text Normalization: Converted all text to lowercase and standardized academic degree names (e.g., merging "B.Tech" and "b. tech" into the uniform token "btech").

Step-4: Cleaning & Filtering: Used regular expressions to strip excessive punctuation and non-textual artifacts, retaining only valid alphanumeric characters and sentence delimiters.

Step-5: Tokenization: Split the continuous text into individual word tokens, discarding extremely rare words (frequency < 2) to form the final high-quality vocabulary.

Dataset Statistics: The final clean corpus consists of approximately **75 documents**, yielding **57,639 total tokens** with a **Vocabulary Size of 3,292 unique words**.

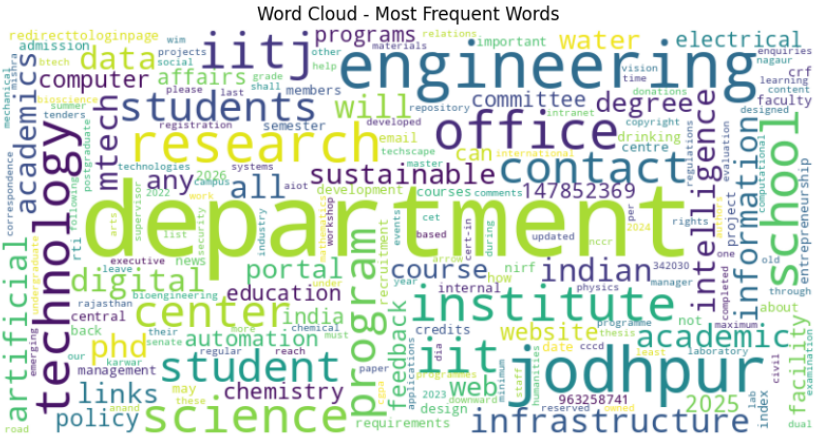

Figure 1: Word Cloud illustrating the most frequent words in the preprocessed IIT Jodhpur corpus.(removed stop words for better visualisation)

2. Word2Vec Training & Hyperparameters

We trained both **Continuous Bag of Words (CBOW)** and **Skip-gram (SG)** models using Negative Sampling. We evaluated semantic coherence by measuring the average cosine similarity of predefined, highly related academic word pairs across various hyperparameters.

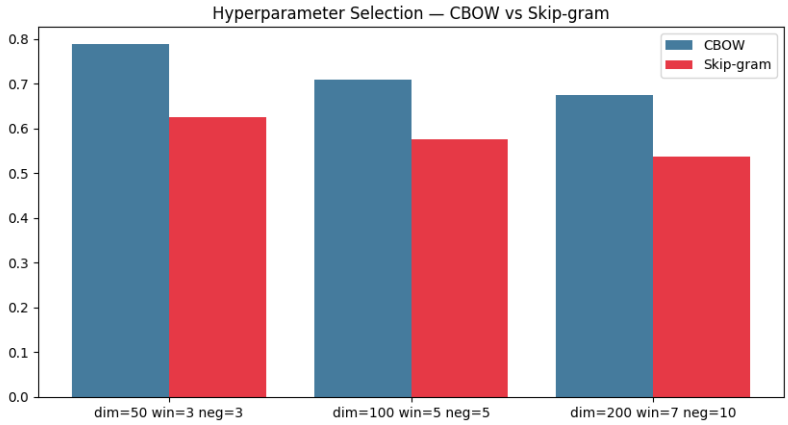

Figure 2: Hyperparameter Probe Scores (Cosine Similarity on known pairs).

Findings: On our relatively small, specialized corpus, smaller embedding dimensions (Dim=50) with tighter context windows (Win=3) and fewer negative samples (Neg=3) outperformed larger dimensions. Massive embedding sizes (Dim=200) led to sparse, overfitted representations, slightly degrading semantic coherence.

3. Implementation: Scratch vs. Gensim

To fully understand the underlying mechanics, we implemented Word2Vec from scratch using pure NumPy before comparing it to the production-grade Gensim library.

- **Scratch Implementation:** We manually coded the forward and backward passes. For Skip-gram with negative sampling, we strictly followed the mathematical objective to maximize the dot product of valid context pairs while minimizing it for randomly drawn negative samples (k):

$$J = -\log \sigma(u_o^T v_c) - \sum_{i=1}^{k} \log \sigma(-u_{n_i}^T v_c)$$

  While mathematically accurate and capable of forming valid semantic clusters, the pure-Python loops scale poorly, processing epochs significantly slower than C-optimized libraries.
- **Gensim Implementation:** The library models leverage highly optimized C-extensions, dynamic context window sizing, and frequent-word subsampling. This allows Gensim to train larger dimensions (e.g., 300-dim) over many epochs in fractions of a second, yielding much smoother and more robust vector spaces.

4. Semantic Analysis

We analyzed the embedding spaces using a highly trained 300-dimensional Gensim model and our scratch models.

Top 5 Nearest Neighbors (Gensim Skip-gram):

- **research:** *xrd, powder, groups, hplc, photoluminescence*
- **student:** *who, certificate, advise, awarded, considered*
- **artificial:** *intelligence, ecosystem, ethics, ai-driven, internships*

- **phd:** *offers, provisionally, mtech, 2026-27, shortlisted*
- **examination:** *comprehensive, viva-voce, first, examiner, proposed*

Analogy Experiments:

1. `ug : btech :: pg : ?` → **mtech**, **msc**. (Highly meaningful; successfully maps the hierarchy of degrees).
2. `btech : course :: phd : ?` → **requirements**, **research**. (Accurately reflects that PhDs focus on independent research rather than standard coursework).
3. `machine : learning :: artificial : ?` → **intelligence**, **data**.

Interesting Analogy Note: The model's ability to solve the `machine : learning :: artificial : intelligence` analogy perfectly demonstrates how well embeddings capture domain-specific, multi-word entities (like the "School of Artificial Intelligence and Data Science") strictly from statistical text co-occurrence, without any hardcoded grammar rules.

5. Feature Space Visualization (t-SNE)

We reduced our embeddings to 2D space to observe clustering behavior. Words were color-coded by their broad semantic categories (Academic, Research, AI).

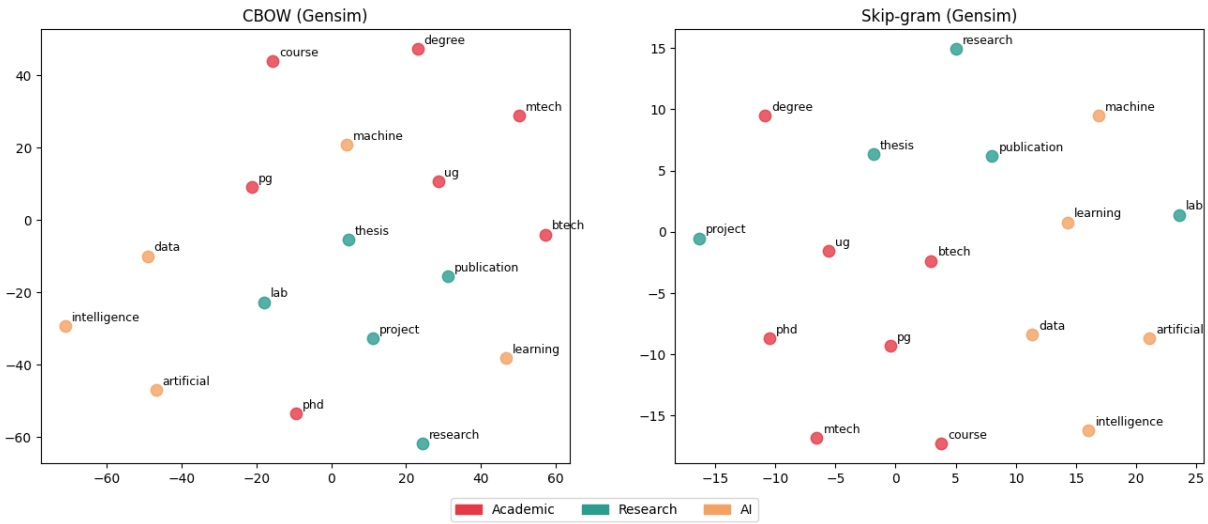

Figure 3: t-SNE visualization of the Gensim embeddings showing distinct semantic clustering.

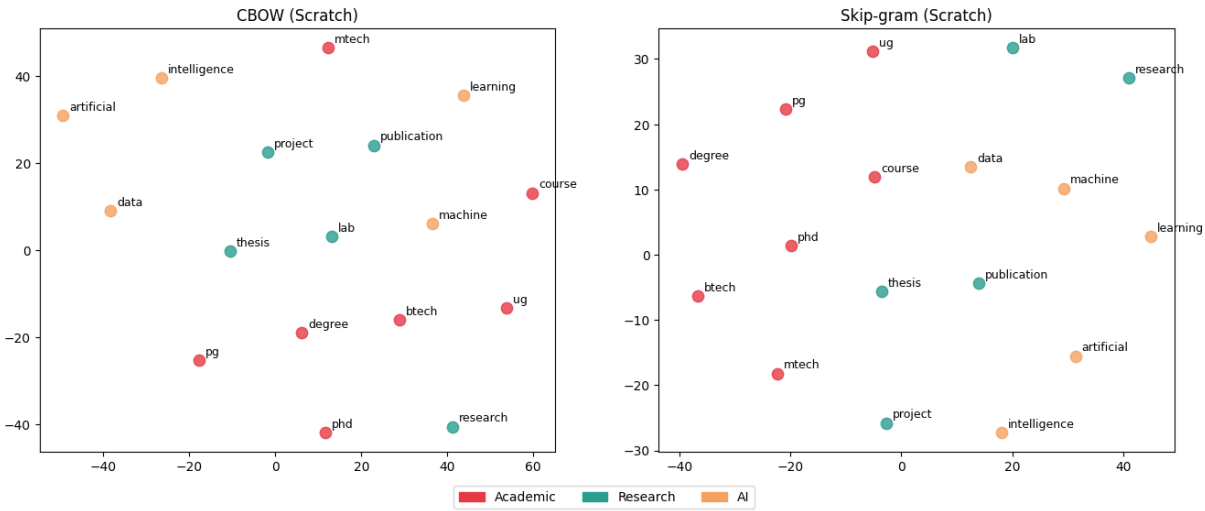

Figure 4: t-SNE visualization of our Scratch embeddings, successfully replicating the topological behavior.

Interpretation of Clustering Behavior:

- **CBOW (Continuous Bag of Words):** Because CBOW predicts a target word by averaging its surrounding context ($\hat{v} = \frac{1}{2m}\sum v_j$), it tends to smooth over rare nuances. In the plots, CBOW forms *tight, dense topical clusters* (e.g., all AI terms clumping closely together).
- **Skip-gram:** Because Skip-gram treats every single target-context pair as an independent observation, it preserves fine-grained semantic details. In the plots, Skip-gram creates a more *spread-out geometry*, separating specific concepts (like 'BTech' vs 'PhD') much more distinctly.

6. Sequence Models: Architecture

We implemented three recurrent architectures from scratch for character-level Indian name generation. All models shared the following hyperparameters: **Embedding Size = 32**, **Hidden Size = 128**, **Batch Size = 32**, **Learning Rate = 0.001**, trained for **50 Epochs**.

1. Vanilla RNN (25,280 Trainable Parameters):
A standard recurrent network that updates a hidden state iteratively via linear transformations. It is lightweight but struggles with long-term character dependencies.

$$
\begin{aligned}
z_t &= W_{ih}x_t + W_{hh}h_{t-1} \\
h_t &= \tanh(z_t) \\
y_t &= W_{ho}h_t
\end{aligned}
$$

2. Bidirectional LSTM (173,534 Trainable Parameters):
Processes sequences forward and backward, maintaining stable cell states (C_t) using gating mechanisms. The input to the gates (z_t) is the concatenation of the hidden state and current input: $z_t = [h_{t-1}, x_t]$.

$$
\begin{aligned}
f_t &= \sigma(W_f z_t + b_f) \\
i_t &= \sigma(W_i z_t + b_i) \\
\tilde{C}_t &= \tanh(W_c z_t + b_c) \\
C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\
h_t &= \sigma(W_o z_t + b_o) * \tanh(C_t)
\end{aligned}
$$

3. RNN with Causal Attention (62,046 Trainable Parameters):
A causal (decoder-only) model where each time-step calculates alignment scores ($e_{j,t}$) against its own past states (h_j) to form a weighted context vector (c_t) before final prediction.

$$
\begin{aligned}
e_{j,t} &= V_{att}^T \tanh(W_q h_t + W_k h_j) \\
\alpha_{j,t} &= \text{softmax}(e_{j,t}) \\
c_t &= \sum \alpha_{j,t} h_j
\end{aligned}
$$

7. Generation Metrics & Qualitative Analysis

We generated 100 novel names using each model to evaluate their ability to capture the phonetic rules of the training dataset.

Model Iteration	Novelty Rate	Diversity Rate
Vanilla RNN	99.00%	100.00%
Causal Attention	96.00%	100.00%
BLSTM	100.00%	100.00%

Realism & Common Failure Modes:

- **Causal Attention:** Produced the most realistic, contextually aware Indian names (*Osbeulent Kumas, Farida Khani Rao, Nitan Pillai*). The self-attention mechanism allowed the model to reliably complete a valid surname even after generating a long first name.
- **Vanilla RNN:** Generated somewhat plausible but significantly noisier names (*Yamish Wagen, Xenxel Bala*). It exhibited classic RNN failure modes, occasionally losing track of the sequence length and generating unpronounceable character strings.
- **BLSTM (Failure Mode):** Generated absolute gibberish (*Thufk vagc, Ub'nkakh*). *Why?* A Bidirectional LSTM requires the *entire* target sequence to exist so it can calculate the backward pass. Autoregressive character generation (predicting the next character one by one) inherently breaks this architecture. Because the "future" characters do not exist yet at inference time, the model is forced to predict without the backward context it heavily relied upon during training, causing a total structural collapse in the output.